



1. backend/server.js — Main Backend Entry Point

```
const express = require('express'); // Import Express framework
const mongoose = require('mongoose'); // Import Mongoose to connect with MongoDB
const cors = require('cors'); // Import CORS to allow cross-origin requests
const itemRoutes = require('./routes/itemRoutes'); // Import API routes
const connectDB = require('./config/db'); // Import MongoDB connection setup
const { errorHandler } = require('./middleware/errorHandler'); // Custom error handling middleware

const app = express(); // Initialize Express app
connectDB(); // Connect to MongoDB

app.use(cors()); // Enable CORS for frontend access
app.use(express.json()); // Parse incoming JSON bodies

app.use('/api/items', itemRoutes); // Mount item-related routes at /api/items

app.use(errorHandler); // Global error handler middleware

app.listen(5000, () => console.log('Server running on 5000')); // Start server
```



2. backend/config/db.js — MongoDB Connection File

```
const mongoose = require('mongoose'); // Import Mongoose library

const connectDB = async () => {
  await mongoose.connect('mongodb://localhost:27017/mern_demo'); // Connect to local MongoDB database
  console.log('MongoDB Connected'); // Log confirmation
};

module.exports = connectDB; // Export function to use in server.js
```



3. backend/models/Item.js — Mongoose Schema

```
const mongoose = require('mongoose'); // Import Mongoose

// Define schema for item collection (just one field: name)
const itemSchema = new mongoose.Schema({
  name: { type: String, required: true } // Field must be a string and
  required
});

// Export model to be used in controllers
module.exports = mongoose.model('Item', itemSchema);
```



4. backend/routes/itemRoutes.js — Express Routes

```
const express = require('express'); // Import Express
const { getItem, addItem } = require('../controllers/itemController'); //
Import controller functions

const router = express.Router(); // Create router object

router.get('/', getItem); // GET /api/items → fetch all items
router.post('/', addItem); // POST /api/items → add a new item

module.exports = router; // Export router to be mounted in server.js
```



5. backend/controllers/itemController.js — Logic Behind Each Route

```
const Item = require('../models/Item'); // Import Mongoose model

// Controller for GET /api/items
const getItem = async (req, res) => {
  const items = await Item.find(); // Fetch all items from DB
  res.json(items); // Send response as JSON
};

// Controller for POST /api/items
const addItem = async (req, res) => {
  const newItem = new Item({ name: req.body.name }); // Create item from
  request body
  await newItem.save(); // Save to database
  res.status(201).json(newItem); // Send back saved item with status 201
};

module.exports = { getItem, addItem }; // Export both functions
```



6. backend/middleware/errorHandler.js — Error Middleware

```
// Custom error handler function
const errorHandler = (err, req, res, next) => {
  console.error(err.stack); // Log error stack trace
  res.status(500).json({ message: err.message }); // Respond with 500 error
  and message
};

module.exports = { errorHandler }; // Export to use in server.js
```

7. `frontend/src/index.js` — React App Entry Point

```
import React from 'react'; // Import React
import ReactDOM from 'react-dom/client'; // React DOM for rendering
import App from './App'; // Import root App component

const root = ReactDOM.createRoot(document.getElementById('root')); // Get
<div id="root">
root.render(<App />); // Render App into the root div
```

8. `frontend/src/App.js` — React Routing Setup

```
import React from 'react'; // Import React
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
// Import routing components
import HomePage from './pages/HomePage'; // Import HomePage

function App() {
  return (
    <Router> {/* Routing system wrapper */}
      <Routes>
        <Route path="/" element={<HomePage />} /> {/* Render HomePage at
"/" */}
      </Routes>
    </Router>
  );
}

export default App; // Export App component
```

9. `frontend/src/pages/HomePage.js` — Main Page Component

```
import React from 'react'; // Import React
import ItemList from '../components/ItemList'; // Import ItemList component

const HomePage = () => {
  return (
```

```

    <div>
      <h1>My Items</h1>
      <ItemList /> {/* Display the item list with form */}
    </div>
  );
};

export default HomePage;

```



10. frontend/src/components/ItemList.js — List + Form Component

```

import React, { useEffect, useState } from 'react'; // Import React hooks
import axios from 'axios'; // Import axios for AJAX

const ItemList = () => {
  const [items, setItems] = useState([]); // State to hold item list
  const [name, setName] = useState(''); // State for input box

  useEffect(() => {
    // Fetch items when component mounts
    axios.get('http://localhost:5000/api/items')
      .then(res => setItems(res.data)); // Save items to state
  }, []);

  const handleAdd = () => {
    // Add item via POST
    axios.post('http://localhost:5000/api/items', { name })
      .then(() => window.location.reload()); // Reload page after adding
  };

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.value)} /> {/*
Input field */}
      <button onClick={handleAdd}>Add Item</button> {/* Add button */}
      <ul>
        {items.map(item => (
          <li key={item._id}>{item.name}</li> // Display each item
        ))}
      </ul>
    </div>
  );
};

export default ItemList;

```



How It All Works Together (Flow Recap):

1. User opens React app → HomePage → ItemList loads.

2. `ItemList` uses `axios.get()` to hit Express `/api/items`.
 3. Express receives request → runs `getItems()` controller → queries MongoDB.
 4. MongoDB returns data → Express responds with JSON → React renders list.
 5. User types name + clicks “Add” → React sends `axios.post()` request.
 6. Express runs `addItem()` controller → stores new item in MongoDB → reloads.
-

Perfect! Let’s now add **authentication (JWT + Google OAuth)** to this minimal MERN stack app, with **step-by-step flow** and **detailed comments**.

✅ JWT Authentication in MERN — Step-by-Step

📁 Folder Structure Updates:

```
backend/
├── controllers/
│   └── authController.js
├── middleware/
│   ├── errorHandler.js
│   └── authMiddleware.js
├── models/
│   └── User.js
├── routes/
│   ├── itemRoutes.js
│   └── authRoutes.js
└── server.js

frontend/
├── pages/
│   └── LoginPage.js
└── components/
    └── ProtectedRoute.js
```

🔒 1. JWT Auth Backend Setup

✅ `models/User.js`

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  password: { type: String }, // Optional if using Google
  googleId: { type: String }, // For Google OAuth
});

module.exports = mongoose.model('User', userSchema);
```

✓ controllers/authController.js

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const User = require('../models/User');

// Generate JWT Token
const generateToken = (id) => {
  return jwt.sign({ id }, 'secret123', { expiresIn: '7d' });
};

// Register New User
const register = async (req, res) => {
  const { email, password } = req.body;

  const existing = await User.findOne({ email });
  if (existing) return res.status(400).json({ message: 'User exists' });

  const hashed = await bcrypt.hash(password, 10);
  const user = await User.create({ email, password: hashed });

  res.json({ token: generateToken(user._id) });
};

// Login User
const login = async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user || !await bcrypt.compare(password, user.password))
    return res.status(400).json({ message: 'Invalid credentials' });

  res.json({ token: generateToken(user._id) });
};

// Google OAuth Login
const googleLogin = async (req, res) => {
  const { email, googleId } = req.body;

  let user = await User.findOne({ email });

  if (!user) {
    user = await User.create({ email, googleId });
  }

  res.json({ token: generateToken(user._id) });
};

module.exports = { register, login, googleLogin };
```

✓ routes/authRoutes.js

```
const express = require('express');
const { register, login, googleLogin } =
  require('../controllers/authController');
const router = express.Router();

router.post('/register', register);
```

```
router.post('/login', login);
router.post('/google', googleLogin);

module.exports = router;
```

✓ middleware/authMiddleware.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Not authorized' });

  try {
    const decoded = jwt.verify(token, 'secret123');
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (err) {
    res.status(401).json({ message: 'Token failed' });
  }
};

module.exports = { protect };
```

✓ Update server.js to include:

```
const authRoutes = require('./routes/authRoutes');
app.use('/api/auth', authRoutes);
```



2. Frontend for JWT + Google OAuth

✓ pages/LoginPage.js

```
import React, { useState } from 'react';
import axios from 'axios';
import { GoogleLogin } from '@react-oauth/google';
import { jwtDecode } from 'jwt-decode';
import { useNavigate } from 'react-router-dom';

const LoginPage = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const navigate = useNavigate();

  const login = async () => {
    const res = await axios.post('http://localhost:5000/api/auth/login',
    { email, password });
    localStorage.setItem('token', res.data.token);
    navigate('/');
  };

  const handleGoogleSuccess = async (credentialResponse) => {
```

```

    const decoded = jwtDecode(credentialResponse.credential);
    const res = await axios.post('http://localhost:5000/api/auth/google', {
      email: decoded.email,
      googleId: decoded.sub
    });
    localStorage.setItem('token', res.data.token);
    navigate('/');
  };

  return (
    <div>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      <input type="password" value={password} onChange={e =>
setPassword(e.target.value)} />
      <button onClick={login}>Login</button>

      <GoogleLogin onSuccess={handleGoogleSuccess} onError={() =>
alert('Google Login Failed')} />
    </div>
  );
};

export default LoginPage;

```

✓ components/ProtectedRoute.js

```

import React from 'react';
import { Navigate } from 'react-router-dom';

const ProtectedRoute = ({ children }) => {
  const token = localStorage.getItem('token');
  return token ? children : <Navigate to="/login" />;
};

export default ProtectedRoute;

```

✓ Update App.js to protect routes

```

import LoginPage from './pages/LoginPage';
import ProtectedRoute from './components/ProtectedRoute';

<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route path="/" element={
    <ProtectedRoute>
      <HomePage />
    </ProtectedRoute>
  } />
</Routes>

```

Flow Summary

JWT Auth Flow:

1. User registers or logs in.
2. Server returns a JWT token.
3. Token is saved to localStorage.
4. ProtectedRoute checks token before allowing access.

Google OAuth Flow:

1. User signs in with Google.
2. Google sends `credential` → React decodes with `jwtDecode()`.
3. React sends `{ email, googleId }` to backend.
4. Backend stores/checks user and responds with JWT.



middleware/authMiddleware.js

```
const jwt = require('jsonwebtoken');           // Import the JWT library
to verify tokens
const User = require('../models/User');        // Import the User model to
fetch user data from DB
```

Purpose of this file:

This middleware function ensures that a user is authenticated before accessing protected routes — i.e., they must provide a valid JWT in the request.



protect Middleware Function

```
const protect = async (req, res, next) => {
```

-  Declares an **asynchronous middleware function**.
-  It will run **before the route handler**, to check if the user has a valid token.

```
const token = req.headers.authorization?.split(' ')[1];
```

What this line does:

- `req.headers.authorization` — this accesses the `Authorization` header from the request.
 - ♦ **Example:** `Authorization: Bearer <token>`
- `?.split(' ')[1]` — this **safely splits** the string `"Bearer <token>"` into an array like:
 - `['Bearer', 'actual_token_here']`

And gets the second item (`[1]`) which is the actual token.

- If the header doesn't exist, it won't crash — `?.` safely returns `undefined`.

```
if (!token) return res.status(401).json({ message: 'Not authorized' });
```

-  If the token is missing, immediately return **401 Unauthorized**.
- This protects the route from being accessed without credentials.

```
try {  
  const decoded = jwt.verify(token, 'secret123');
```

-  Try to **verify and decode** the token using the secret key 'secret123'.
- If the token is valid, this gives:
- `decoded = { id: "userIdHere", iat: ..., exp: ... }`

```
req.user = await User.findById(decoded.id).select('-password');
```

-  This line **fetches the user's data from MongoDB** using the decoded user ID.
- `.select('-password')` excludes the password field from the result for security.
-  Attaches the authenticated user's info to the `req` object → `req.user`
So now, any route using this middleware can access `req.user`.

```
next(); // Move to next middleware or route handler
```

-  If everything is valid, this moves the request to the next middleware or actual route logic.
- Example: the protected controller can now access `req.user`.

```
} catch (err) {  
  res.status(401).json({ message: 'Token failed' });  
}  
};
```

-  If token verification fails (expired, invalid, etc.), catch the error and return 401 Unauthorized.

```
module.exports = { protect };
```

-  Export the middleware so you can use it in your routes like this:
 - `router.get('/secure-data', protect, someController)`
-



Summary

Line	Purpose
<code>authorization?.split()</code>	Extracts the actual JWT token from header
<code>jwt.verify()</code>	Decodes and verifies the token using a secret key

Line	Purpose
<code>User.findById()</code>	Fetches user info from MongoDB
<code>req.user = ...</code>	Attaches the user to the request object
<code>next()</code>	Allows request to move to the actual route
<code>401 responses</code>	Rejects access if token is invalid or missing

Excellent — now let's walk through how to **use the `protect` middleware in backend routes**, and how the **frontend sends the token** to access protected data. I'll give you both the backend and frontend flow clearly.

✓ 1. How to Use `protect` Middleware in Backend Routes

🔒 Step 1: Apply `protect` to Protected Routes

Let's say you want to **protect** the `/api/items` routes so only logged-in users can use them:

✓ `routes/itemRoutes.js`

```
const express = require('express');
const { getItems, addItem } = require('../controllers/itemController');
const { protect } = require('../middleware/authMiddleware');

const router = express.Router();

// 📌 Now protected – user must send token to access
router.get('/', protect, getItems);
router.post('/', protect, addItem);

module.exports = router;
```

✓ `protect` runs **before** `getItems` or `addItem`

🔒 If the token is missing or invalid → user gets 401 Unauthorized

✓ 2. How Frontend Sends JWT to Backend

Once the user logs in and gets a token, it's stored in `localStorage`. We send this token in every request that hits protected routes.

✓ React Frontend Flow (using `axios`)

Let's update the **GET and POST request** in `ItemList.js` to send the token.

frontend/src/components/ItemList.js (Updated)

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const ItemList = () => {
  const [items, setItems] = useState([]);
  const [name, setName] = useState('');

  const token = localStorage.getItem('token'); // 🔑 Get token from storage

  useEffect(() => {
    // 📡 Send token in Authorization header for protected route
    axios.get('http://localhost:5000/api/items', {
      headers: {
        Authorization: `Bearer ${token}`
      }
    }).then(res => setItems(res.data));
  }, []);

  const handleAdd = () => {
    axios.post('http://localhost:5000/api/items', { name }, {
      headers: {
        Authorization: `Bearer ${token}`
      }
    }).then(() => window.location.reload());
  };

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.value)} />
      <button onClick={handleAdd}>Add Item</button>
      <ul>
        {items.map(item => <li key={item._id}>{item.name}</li>)}
      </ul>
    </div>
  );
};

export default ItemList;
```

Full Request Flow (Step-by-Step)

✅ When using JWT Auth

- ✅ User logs in on frontend → receives a JWT from backend
- ✅ JWT is saved in `localStorage`
- ✅ When React loads protected page (`ItemList`):
 - It grabs the JWT from `localStorage`
 - Sends it in Authorization header:
`Authorization: Bearer <your_token>`
- ✅ Express backend:
 - `protect` middleware reads and verifies the token

- If valid → attaches `req.user` and calls the controller (e.g. `getItems`)
- If invalid → sends 401 Unauthorized

Extra Tip: Automatically Attach Token (Optional)

Instead of adding headers in every axios call, you can set a global default once:

✓ In `src/index.js` OR `App.js`:

```
axios.defaults.headers.common['Authorization'] = `Bearer ${localStorage.getItem('token')}`;
```

Then you don't need to add headers manually in each request.

BONUS: Accessing `req.user` in Controller

You can now use `req.user` in your controller like this:

✓ `controllers/itemController.js`

```
const getItems = async (req, res) => {  
  console.log(req.user); //  Shows current logged-in user's data  
  const items = await Item.find();  
  res.json(items);  
};
```

Perfect — here's a complete and clear guide covering the **next-level authentication features** for your MERN app:

✓ 1. Secure Logout Flow

Concept:

Since we're using JWT (stateless), logout is **frontend-only**:

- ✓ Remove the token from `localStorage`
- ✓ Redirect to login page

✓ Logout Button in React:

```
const handleLogout = () => {
```

```
localStorage.removeItem('token'); // 🗑️ Remove token
```

```
window.location.href = '/login'; // 🔄 Redirect
```

```
};
```

🔒 There's no backend logout with JWT since the token isn't stored server-side. To enhance this, you can implement **token blacklisting** on backend, but that's optional.

🔄 2. Refresh Token Implementation

🧠 Problem:

JWTs expire (e.g., in 15 mins) and the user would be logged out. We fix this using:

- 🔑 Short-lived **access token**
 - 🔄 Long-lived **refresh token** (stored in secure cookie)
-

✅ Backend Flow:

Step 1: When logging in, issue two tokens:

```
const accessToken = jwt.sign({ id }, 'accessSecret', { expiresIn: '15m' });
```

```
const refreshToken = jwt.sign({ id }, 'refreshSecret', { expiresIn: '7d' });
```

```
res.cookie('refreshToken', refreshToken, {  
  httpOnly: true, secure: true, sameSite: 'strict'  
});
```

```
res.json({ accessToken });
```

✅ /api/auth/refresh Route:

```
router.post('/refresh', (req, res) => {
```

```
  const refreshToken = req.cookies.refreshToken;
```

```
if (!refreshToken) return res.sendStatus(401);
```

```
jwt.verify(refreshToken, 'refreshSecret', (err, decoded) => {
```

```
  if (err) return res.sendStatus(403);
```

```
  const newAccessToken = jwt.sign({ id: decoded.id }, 'accessSecret', { expiresIn: '15m' });
```

```
  res.json({ accessToken: newAccessToken });
```

```
});
```

```
});
```

✅ Frontend Axios Refresh Logic:

Use an interceptor to auto-refresh tokens:

```
axios.interceptors.response.use(
```

```
  res => res,
```

```
  async err => {
```

```
    const original = err.config;
```

```
    if (err.response.status === 401 && !original._retry) {
```

```
      original._retry = true;
```

```
      const res = await axios.post('/api/auth/refresh', {}, { withCredentials: true });
```

```
      localStorage.setItem('token', res.data.accessToken);
```

```
      axios.defaults.headers.common['Authorization'] = `Bearer ${res.data.accessToken}`;
```

```
      return axios(original);
```

```
    }
```

```
    return Promise.reject(err);
```

```
  }
```

```
);
```

3. Admin-Only Route Protection (protectAdmin)

Idea:

Allow access only to users with role: 'admin'.

Extend your User model:

```
const userSchema = new mongoose.Schema({  
  email: String,  
  password: String,  
  role: { type: String, default: 'user' } // Add role field  
});
```

Add protectAdmin.js middleware:

```
const protectAdmin = (req, res, next) => {  
  if (req.user?.role !== 'admin') {  
    return res.status(403).json({ message: 'Admin access only' });  
  }  
  next();  
};
```

```
module.exports = { protectAdmin };
```

Use in Routes:

```
const { protect } = require('./authMiddleware');  
const { protectAdmin } = require('./protectAdmin');
```

```
router.get('/admin/dashboard', protect, protectAdmin, adminController);
```

● 4. Google OAuth + Token-Based Protection

✓ How it Works:

1. User signs in with Google.
 2. Frontend gets credential from Google → sends to backend.
 3. Backend verifies Google ID + issues JWT.
 4. From now on, **that JWT is used** to access protected routes — same as normal users.
-

✓ Frontend Flow:

<GoogleLogin

```
onSuccess={async (res) => {  
  
  const decoded = jwtDecode(res.credential); // Get email, sub  
  
  const r = await axios.post('/api/auth/google', {  
  
    email: decoded.email,  
  
    googleId: decoded.sub  
  
  });  
  
  localStorage.setItem('token', r.data.token); // ✓ Save JWT  
  
}}  
  
>
```

✓ Backend Route Recap:

```
const googleLogin = async (req, res) => {  
  
  const { email, googleId } = req.body;  
  
  let user = await User.findOne({ email });
```

```
if (!user) {  
  
  user = await User.create({ email, googleId, role: 'user' });  
  
}  
  
const token = jwt.sign({ id: user._id }, 'secret123', { expiresIn: '7d' });  
  
res.json({ token });  
  
};
```

✅ After login, user behaves exactly like a normal JWT user.

Summary Table

Feature	Backend Action	Frontend Action
Logout	N/A (token is stateless)	localStorage.removeItem('token')
Refresh Token	Create refresh token (HTTP-only cookie)	Axios interceptor refreshes token
Admin Protect	Check req.user.role === 'admin'	Just include token like normal
Google OAuth	Map Google ID to user → issue JWT	Get credential, decode, send to BE
